



ASTROFY

Guia do usuário

Do pedido da página à implementação validada

Setup, entrevista, especificação, plano técnico, execução e checkup em projetos Astro.

Edição v0.6.0

Gerado em 18 de setembro de 2026

Fonte editorial: docs/user/

Sumário

Astrofy: guia do usuário	4
Percurso recomendado	4
Formatos	4
Visão geral	5
Artefatos do projeto	5
Instalação e setup	7
Requisitos	7
Instalação global	7
Primeira execução	7
Resultado esperado	8
Planejamento de uma página	9
Pontos de revisão	9
Entrevista e conteúdo	10
Informações comuns	10
Como responder	10
Estado retomável no CLI	10
Exemplo curto	11
Plano de implementação	12
Aprovação	12
Execução e validação	13
Conferência mínima	13
Exemplos por tipo de página	14
Landing page de vendas	14
Página de produto	14
Página de serviço	14
Página inicial	14
Página Sobre	15
Página de contato	15
Página de preços	15
Resultado comum	16
Execução retomável	17
Estados da tarefa	17
Passagem prática	17

Retomada em outra sessão	18
Retomada e manutenção	19
Atualização do Astrofy	19
Solução de problemas	20
O projeto não é reconhecido	20
O setup encontrou JSON inválido	20
O tipo de página ficou ambíguo	20
A implementação divergiu do conteúdo aprovado	20
O plano ficou desatualizado	20
O Chromium não está disponível	20
A migração pendente deve falhar no CI	20
O ebook não confere	21

Astrofy: guia do usuário

Este manual mostra como usar o Astrofy para entender um projeto Astro, definir uma página por conversa, transformar o conteúdo aprovado em um plano técnico e acompanhar a implementação.

Percurso recomendado

1. Entenda o fluxo completo.
2. Instale e execute o setup.
3. Escolha e planeje o tipo de página.
4. Responda à entrevista de conteúdo.
5. Gere o plano de implementação.
6. Implemente e valide.
7. Compare exemplos dos sete tipos de página.
8. Acompanhe os estados de uma tarefa.
9. Retome ou altere um trabalho.
10. Consulte as soluções para falhas comuns.

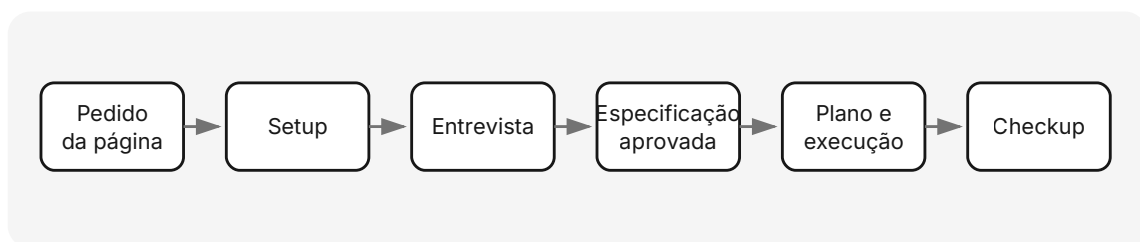
Formatos

Os capítulos deste diretório são a fonte canônica. O projeto gera a mesma edição em PDF e EPUB dentro de `ebook/`. A versão do manual acompanha a versão do pacote Astrofy.

Visão geral

O Astrofy organiza a criação e a evolução de sites Astro em quatro fases:

1. `astrofy-setup` lê o projeto, reconhece seus recursos e prepara o estado de trabalho em `.astrofy/`.
2. `astrofy-page-planner` identifica o tipo de página e encaminha a conversa para uma especialista.
3. `astrofy-implementation-planner` relaciona o conteúdo aprovado com rotas, componentes, integrações, testes e documentação do projeto real.
4. As skills técnicas implementam e conferem cada parte do plano.

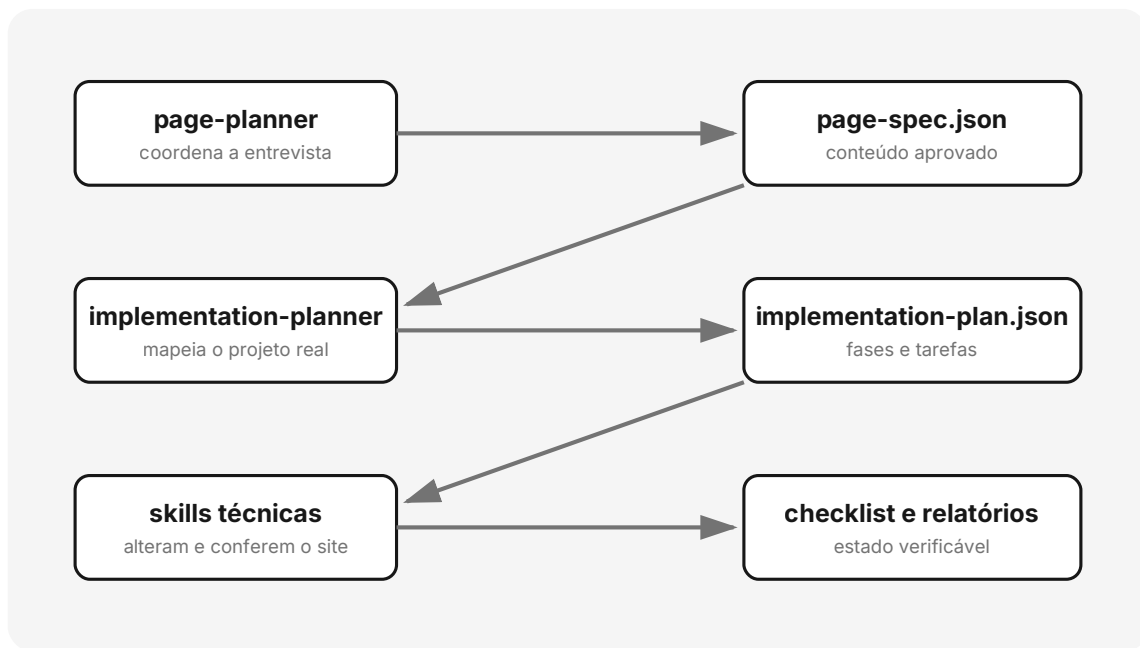


Fluxo do pedido ao checkup

O Astrofy não exige um formulário completo no primeiro contato. A pessoa pode começar com uma frase, como “quero uma página para meu produto”. A orquestradora confirma o tipo, coleta o conteúdo por etapas e apresenta três alternativas quando uma escolha editorial depende do usuário.

Artefatos do projeto

Caminho	Conteúdo
<code>.astrofy/config/project.json</code>	Configuração reconhecida pelo setup
<code>.astrofy/pages/<slug>/page-spec.json</code>	Especificação estruturada da página
<code>.astrofy/pages/<slug>/page.md</code>	Leitura humana do conteúdo coletado
<code>.astrofy/plans/<slug>/implementation-plan.json</code>	Fases e tarefas
<code>.astrofy/plans/<slug>/implementation-plan.md</code>	Plano técnico para revisão
<code>astrofy.checklist.json</code>	Estado das verificações do site



Relação entre orquestradoras, artefatos e skills

Instalação e setup

Requisitos

- Node.js 22.12.0 ou posterior.
- Um projeto Astro existente com `package.json`.
- Permissão de escrita no diretório do projeto.

Instalação global

```
npm install --global @promovaweb/astrofy
astrofy --version
```

Para manter a versão junto ao projeto:

```
npm install --save-dev @promovaweb/astrofy
npx astrofy --version
```

O Chromium é opcional e fica no cache do Playwright, fora do pacote npm:

```
astrofy browser status
astrofy browser install
```

Instale-o quando o projeto usar as verificações reais de renderização.

Primeira execução

Comece pela skill `astrofy-setup`. Ela lê `package.json`, a configuração do Astro, as rotas, os componentes, as coleções de conteúdo e as integrações já instaladas. Antes de gravar arquivos, confira a leitura do CLI:

```
astrofy inspect --root .
astrofy init --root . --dry-run
astrofy init --root .
```

O setup preserva convenções válidas do projeto. Diretórios personalizados, adaptadores e integrações detectadas entram no contexto usado pelas próximas skills.

Resultado esperado

Ao terminar, o projeto possui `.astrofy/`, configuração válida e um registro inicial da checklist. Execute `git diff` para revisar os arquivos criados.

Planejamento de uma página

Chame `astrofy-page-planner` com o pedido em linguagem natural. A skill lê o contexto preparado pelo setup e escolhe uma especialista.

Pedido	Especialista
Landing page de vendas	<code>astrofy-plan-sales-page</code>
Página de produto	<code>astrofy-plan-product-page</code>
Página de serviço	<code>astrofy-plan-service-page</code>
Página inicial	<code>astrofy-plan-homepage</code>
Página Sobre	<code>astrofy-plan-about-page</code>
Página de contato	<code>astrofy-plan-contact-page</code>
Página de preços	<code>astrofy-plan-pricing-page</code>

Se o pedido combinar dois formatos, a orquestradora define um tipo principal e registra as necessidades complementares. Uma página de produto com tabela de planos, por exemplo, continua sendo produto e recebe requisitos de preços.

Pontos de revisão

A conversa avança por finalidade, público, arquitetura, conteúdo, mídia, comportamento e revisão integral. Cada etapa dependente de preferência oferece três opções materialmente diferentes, com uma recomendação fundamentada.

A saída segue `packages/schemas/page-spec.schema.json`. A implementação ainda não começa: primeiro a pessoa revisa `page.md` e aprova a especificação.

Entrevista e conteúdo

A especialista pergunta somente o necessário para o tipo escolhido. Ela aceita texto pronto, notas incompletas ou autorização para propor alternativas.

Informações comuns

- finalidade e ação principal;
- público e contexto de chegada;
- áreas da página e ordem de leitura;
- títulos, textos, rótulos e mensagens de estado;
- imagens, vídeos, ilustrações e suas fontes;
- links, formulários e integrações;
- fatos que precisam de fonte;
- conteúdo ainda pendente.

Como responder

Você pode responder uma etapa por vez. Use `aprovar`, `ajustar` ou `manter pendente` para cada proposta. Quando fornecer um texto final, diga que ele deve ser preservado. Quando quiser ajuda editorial, informe os fatos disponíveis e o tom desejado.

Áreas opcionais podem ficar pendentes. A aprovação final registra seu estado e impede que uma lacuna seja preenchida silenciosamente durante a implementação.

Estado retomável no CLI

Inicie a entrevista informando o tipo da página:

```
astrofy page create --slug atlas --type product
astrofy page status --slug atlas
```

A saída apresenta até três itens em `nextQuestions`. Registre cada resposta com o identificador recebido:

```
astrofy page answer --slug atlas --field title --value "Atlas"
astrofy page answer --slug atlas --field route --value /atlas/
```

O arquivo `interview.json` conserva respostas e lacunas. Quando todas as entradas obrigatórias existem, o CLI gera `page-spec.json` com conteúdo, SEO, integrações, acessibilidade e testes.

Exemplo curto

Usuário: Quero uma página do produto Atlas.

Astrofy: A ação principal deve iniciar teste, solicitar demonstração ou comprar?

Usuário: Solicitar demonstração.

Astrofy: Vou propor três arquiteturas para uma página de produto orientada à demo.

Plano de implementação

Depois da aprovação da página, chame `astrofy-implementation-planner`. A skill relê o projeto para relacionar cada área com arquivos existentes e mudanças necessárias.

O plano registra:

- rota e modo de renderização;
- layouts, componentes e conteúdo reutilizados;
- arquivos que serão criados ou alterados;
- integrações e variáveis de ambiente;
- acessibilidade, SEO e dados estruturados;
- testes automáticos e revisões manuais;
- documentação afetada;
- dependências, ordem e estimativa de cada tarefa.

O JSON segue `packages/schemas/implementation-plan.schema.json`. O Markdown equivalente permite revisar o plano sem ferramentas adicionais.

Gere o plano estruturado depois que a entrevista estiver pronta:

```
astrofy plan --slug atlas
```

As tarefas começam como `pending` ou `ready`. Dependências concluídas liberam a próxima tarefa sem apagar o histórico do plano.

Aprovação

Confira se cada tarefa aponta para um resultado observável e uma forma de conferência. Ajuste o plano antes de executar quando o caminho proposto não combinar com a arquitetura atual. A aprovação libera as skills técnicas por fase.

Execução e validação

O plano indica as skills adequadas para cada tarefa. Uma sequência comum usa:

1. `astrofy-architecture`, `astrofy-routing` e `astrofy-config` para a base.
2. `astrofy-components`, `astrofy-sections` e `astrofy-page-design` para a interface.
3. `astrofy-images`, `astrofy-forms` ou `astrofy-react` conforme a página.
4. `astrofy-accessibility`, `astrofy-seo`, `astrofy-open-graph` e `astrofy-structured-data` para qualidade pública.
5. `astrofy-testing`, `astrofy-documentation` e `astrofy-checkup` para fechar o trabalho.

Execute somente as skills relacionadas ao escopo. Cada uma deve ler o plano, conferir a implementação atual, registrar o que alterou e atualizar o estado da tarefa.

Use `apply` para registrar a passagem de estado antes e depois da execução da skill indicada:

```
astrofy apply --slug atlas
astrofy apply --slug atlas --task route --status completed
```

Conferência mínima

```
npm run check
npm run build
astrofy check --root .
astrofy check --root . --browser
```

O primeiro `check` não abre navegador. A opção `--browser` inclui Chromium e confere temas, overflow, responsividade e erros de runtime no preview local.

Use também os testes próprios do projeto e revise a página no navegador em larguras de celular e desktop. Formulários, menus, foco, links e mensagens de erro exigem interação real.

Exemplos por tipo de página

Os exemplos abaixo mostram o pedido, as respostas centrais, a arquitetura aprovada e o foco do plano técnico. Nomes e textos são ilustrativos.

Landing page de vendas

Pedido: "Quero vender a formação Operação DevOps."

Respostas centrais: profissionais de desenvolvimento; inscrição como ação principal; turma com data e vagas; depoimentos autorizados; pagamento externo.

Arquitetura: hero com oferta, transformação esperada, programa, formato, instrutor, depoimentos, preço, perguntas frequentes e chamada final.

Plano técnico: rota da campanha, componentes de prova e preço, integração do checkout, eventos da ação principal, metadados e testes do percurso.

Página de produto

Pedido: "Quero criar uma página para o Atlas, nosso produto de atendimento."

Respostas centrais: equipes de suporte; demonstração como ação principal; capturas reais do produto; integrações confirmadas pela documentação.

Arquitetura: hero, problemas atendidos, casos de uso, recursos, demonstração visual, integrações, perguntas frequentes e chamada final.

Plano técnico: rota `/atlas/`, seções reutilizáveis, formulário de demonstração, imagens responsivas, `SoftwareApplication` em JSON-LD e testes.

Página de serviço

Pedido: "Preciso apresentar nossa consultoria de modernização."

Respostas centrais: empresas com aplicações legadas; diagnóstico como ação; escopo por etapas; entregas e responsabilidades explícitas; contato comercial.

Arquitetura: cenário atendido, resultados, entregas, processo, requisitos do cliente, equipe, casos relacionados, perguntas e contato.

Plano técnico: composição com componentes existentes, formulário com qualificação, confirmação de envio, metadados `Service` e teste dos estados.

Página inicial

Pedido: "Quero reorganizar a Home para explicar a empresa."

Respostas centrais: três públicos; produto como destino principal; serviços e conteúdos como rotas secundárias; mensagem institucional já aprovada.

Arquitetura: proposta central, caminhos por público, produto em destaque, serviços, recursos recentes, comprovação institucional e contato.

Plano técnico: revisão da rota inicial, navegação, seções compartilhadas, ordem responsiva, links internos, metadados da organização e regressão visual.

Página Sobre

Pedido: "Quero uma página Sobre que conte nossa trajetória."

Respostas centrais: fatos documentados; marcos com datas; pessoas com autorização; princípios descritos por ações; contato para imprensa.

Arquitetura: atuação atual, história, marcos, pessoas, princípios, presença pública e canais institucionais.

Plano técnico: conteúdo estruturado, imagens com texto alternativo, schema Organization, links para fontes públicas e revisão de todas as afirmações.

Página de contato

Pedido: "Quero separar suporte de oportunidades comerciais."

Respostas centrais: dois destinos; campos mínimos por assunto; consentimento; prazo de retorno informado; email como canal alternativo.

Arquitetura: escolha do assunto, formulário adaptado, orientações de privacidade, mensagens de envio, prazo de retorno e canais alternativos.

Plano técnico: validação no cliente e servidor, proteção contra abuso, integrações por ambiente, foco após envio, testes de falha e documentação.

Página de preços

Pedido: "Quero comparar três planos do produto."

Respostas centrais: cobrança mensal e anual; limites por plano; impostos e moeda explícitos; teste gratuito; contato para volume maior.

Arquitetura: contexto da cobrança, seletor de período, cartões dos planos, tabela comparativa, condições comerciais, perguntas e ação por plano.

Plano técnico: fonte única para preços, seletor acessível, URLs do checkout, dados estruturados compatíveis com o conteúdo e testes das combinações.

Resultado comum

Cada exemplo termina com `page-spec.json`, `page.md`, `implementation-plan.json` e `implementation-plan.md`. A implementação somente usa fatos e textos aprovados na entrevista.

Execução retomável

Cada tarefa do plano possui identificador estável, dependências, status, arquivos, passos e validações. O agente altera o status junto com o trabalho, para que a próxima sessão encontre o ponto correto de retomada.

Estados da tarefa

Estado	Significado	Próxima ação permitida
pending	Depende de outra tarefa ou informação	Resolver a dependência
ready	Pode começar	Conferir arquivos e iniciar
in_progress	Há trabalho em andamento	Continuar passos registrados
completed	Resultado e validações foram conferidos	Liberar dependentes
failed	Uma validação falhou	Corrigir e executar novamente
skipped	Saiu do escopo com justificativa	Manter a justificativa

Passagem prática

Uma tarefa começa assim:

```
{
  "id": "product-route",
  "status": "ready",
  "dependsOn": [],
  "files": ["src/pages/produtos/atlas.astro"],
  "validations": ["npm run check", "npm run build"]
}
```

Ao iniciar, a skill relê a rota, registra os arquivos observados e muda o status para `in_progress`. Após editar, executa todas as validações listadas e faz a revisão manual indicada pelo plano. Somente então registra:

```
astrofy apply --slug atlas --task product-route --status in_progress
```

```
{
  "id": "product-route",
  "status": "completed",
  "completedAt": "2026-09-18T16:00:00.000Z",
  "validationResults": [
    { "command": "npm run check", "status": "passed" },
    { "command": "npm run build", "status": "passed" }
  ]
}
```

Se um comando falhar, o status passa para `failed` e recebe a mensagem útil para a correção. A tarefa dependente continua pendente. Uma nova execução volta à tarefa com falha antes de avançar.

```
astrofy apply --slug atlas --task product-route --status failed
astrofy apply --slug atlas --task product-route --status completed
```

Ao concluir uma tarefa, o CLI muda para `ready` as dependências que ficaram livres. Sem `--task`, `astrofy apply --slug atlas` devolve a próxima tarefa que pode começar.

Retomada em outra sessão

1. Confira `git status` e os arquivos alterados.
2. Execute `astrofy page status --slug atlas` e leia `page-spec.json`.
3. Execute `astrofy apply --slug atlas --dry-run` para localizar a próxima tarefa.
4. Localize tarefas `in_progress` ou `failed`.
5. Confirme se os arquivos ainda correspondem ao registro.
6. Continue a tarefa e repita suas validações.
7. Atualize o estado pelo CLI e libere as dependências concluídas.

Retomada e manutenção

Os arquivos em `.astrofy/` permitem continuar o trabalho em outra sessão. Ao retomar, execute o setup, leia a especificação e o plano, confira o Git e continue pela primeira tarefa pendente cujas dependências estejam concluídas.

Se o conteúdo mudar antes da implementação, volte à especialista da página, atualize a especificação e gere novamente as partes afetadas do plano. Se a mudança surgir durante a execução, registre seu impacto antes de editar código.

Atualização do Astrofy

```
npm update --save-dev @promovaweb/astrofy
npx astrofy --version
npx astrofy inspect --root .
```

Leia o `CHANGELOG.md` entre a versão instalada e a nova edição. Execute as validações do projeto depois da atualização.

Solução de problemas

O projeto não é reconhecido

Execute `astrofy inspect --root <diretório>` apontando para a pasta que contém o `package.json` do site. Confira se `astro` aparece em `dependencies` ou `devDependencies`.

O setup encontrou JSON inválido

Abra o caminho informado, corrija a sintaxe e repita `astrofy init --dry-run`. O comando preserva arquivos existentes e interrompe a gravação quando não pode interpretá-los.

O tipo de página ficou ambíguo

Informe a finalidade principal e a ação esperada. A orquestradora usa essas duas respostas para escolher a especialista e mantém necessidades secundárias na especificação.

A implementação divergiu do conteúdo aprovado

Pare a tarefa, compare a página com `brief.md` e `page-spec.json` e corrija a implementação. Mudanças editoriais precisam voltar à etapa de conteúdo.

O plano ficou desatualizado

Execute `astrofy plan --slug <slug>` após reler o projeto. O comando recria as tarefas a partir do contrato atual da página. Preserve no Git o plano anterior quando precisar consultar o histórico das tarefas concluídas.

O Chromium não está disponível

Consulte a instalação controlada pelo Astrofy:

```
astrofy browser status
astrofy browser install
```

Depois execute `astrofy check --browser`. No CI, mantenha o cache indicado no workflow distribuído para evitar baixar o Chromium a cada execução.

A migração pendente deve falhar no CI

Use o modo de conferência:

```
astrofy migrate --check
```

O comando retorna código 1 quando o contrato precisa de migração e código 0 quando a configuração já usa a versão atual.

O ebook não confere

Na raiz do Astrofy, execute:

```
npm run ebook  
npm run ebook:verify
```

O primeiro comando recompila PDF, EPUB e manifesto. O segundo compara a edição com todos os capítulos e arquivos do pipeline.